

TP — Aprendizagem Contínua

Disciplina de Otimização Não Linear

Entrega: Código, Relatório curto, Slides

29 de outubro de 2025

1 Visão geral

O *Continual Learning* (ou aprendizado contínuo) é um paradigma de aprendizado de máquina que busca desenvolver modelos capazes de aprender de forma sequencial, incorporando novos conhecimentos sem esquecer o que foi previamente aprendido. Diferentemente do aprendizado tradicional, em que o modelo é treinado uma única vez sobre um conjunto fixo de dados, o aprendizado contínuo enfrenta o desafio do *catastrophic forgetting*, isto é, a perda drástica de desempenho em tarefas anteriores ao aprender novas. Para mitigar esse problema, diversas abordagens têm sido propostas, incluindo regularização dos parâmetros, uso de memória exemplar e arquiteturas dinâmicas que se adaptam ao longo do tempo.

Ao treinar sequencialmente em uma tarefa A e depois em B, modelos tendem a esquecer A. Estudaremos o *trade-off* estabilidade-plasticidade em duas etapas complementares:

- **Parte 1** (minimização): penalização fixa de deriva para controlar o esquecimento.
- **Parte 2** (min-max): penalização adaptativa via jogo entre parâmetros (abordagem min-max) e um multiplicador λ .

1.1 Dados e modelo

Split-MNIST: neste experimento, o conjunto MNIST é dividido em duas tarefas disjuntas: $A = \{0-4\}$ e $B = \{5-9\}$. Cada tarefa é tratada como um problema de classificação independente, permitindo avaliar a capacidade do modelo em reter o conhecimento adquirido na tarefa A após o treinamento sequencial na tarefa B.

O modelo pode ser um *Multi-Layer Perceptron* (MLP) simples, com uma ou duas camadas ocultas e ativação ReLU, ou uma pequena *Convolutional Neural Network* (CNN) adequada à natureza visual do MNIST. A função de perda utilizada é a *cross-entropy*, e a otimização é realizada via *stochastic gradient descent* (SGD) ou Adam. Durante o treinamento sequencial, são registradas métricas de desempenho nas duas tarefas para avaliar o grau de esquecimento (*forgetting*) e a capacidade de transferência (*transfer*).

Notação

Seja θ o vetor de parâmetros atuais do modelo e θ_A a solução obtida após o treinamento completo na tarefa A. A quantidade abaixo mede o quanto os parâmetros se afastaram da solução anterior, caracterizando a **deriva dos pesos** ao longo do aprendizado contínuo:

$$F_{\text{drift}}(\theta; \theta_A) = \frac{1}{2} \|\theta - \theta_A\|_2^2.$$

Essa medida serve tanto como diagnóstico quanto como termo de regularização em abordagens que buscam equilibrar *plasticidade* (adaptação à nova tarefa) e *estabilidade* (retenção do conhecimento anterior).

2 Desenvolvimento

Parte 1 — Trade-off com penalização fixa (sem min-max)

Objetivo. Quantificar e visualizar o conflito entre aprender B e reter A, variando a intensidade da penalização.

Formulação de otimização

Após treinar A e salvar θ_A , otimize na Tarefa B:

$$\min_{\theta} \mathcal{L}_B(\theta) + \lambda F_{\text{drift}}(\theta; \theta_A),$$

com grade inicial $\lambda \in \{0, 10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}\}$.

Procedimento

1. **Treino em A** até um número fixo de épocas ou acurácia-alvo; salve θ_A e separe um conjunto de validação $\mathcal{V}_A$.
2. **Treino em B** para cada λ , usando o mesmo otimizador e esquema de treino.
3. **Rastreio por iteração/época:**

- Acurácia em A e em B.
- Deriva $\|\theta_t - \theta_A\|_2$.
- Esquecimento em A: $\Delta \text{acc}_A = \text{acc}_A^{\text{após A}} - \text{acc}_A^{\text{após B}}$.

Plots obrigatórios

- Curvas $\text{acc}(\mathbf{A})$ e $\text{acc}(\mathbf{B})$ vs. iteração.
- Deriva $\|\theta_t - \theta_A\|_2$ vs. iteração.
- Trade-off: pontos finais de cada λ no plano Esquecimento (x) vs. $\text{acc}(\mathbf{B})$ (y).

Baseline avançado obrigatório (escolher um)

EWC. Importâncias por parâmetro após A:

$$F_i \approx \frac{1}{|\mathcal{D}_A|} \sum_{(x,y) \in \mathcal{D}_A} \left(\frac{\partial}{\partial \theta_i} \ell(f_\theta(x), y) \right)^2 \Big|_{\theta=\theta_A}, \quad \mathcal{L}_B(\theta) + \lambda \sum_i F_i (\theta_i - \theta_{A,i})^2.$$

Synaptic Intelligence (SI). Durante A, acumule $\omega_i \leftarrow \omega_i + \Delta \theta_i \cdot g_i$. Ao final, $\Omega_i = \frac{\omega_i}{(\Delta \theta_i)^2 + \xi}$, $\xi \approx 10^{-3}$, e use $\mathcal{L}_B(\theta) + \lambda \sum_i \Omega_i (\theta_i - \theta_{A,i})^2$.

Relatório curto — Parte 1

Explique a objetiva com λ fixo, discuta por que há conflito entre aprender B e reter A, e interprete os gráficos e a fronteira Pareto-like.

Parte 2 — Min-max com passos alternados e estabilização

Objetivo. Substituir o λ fixo por um jogo adaptativo:

$$\min_{\theta} \max_{\lambda \geq 0} \mathcal{L}_B(\theta) - \lambda F(\theta; \theta_A),$$

com $F = F_{\text{drift}}$ para começar, passos alternados em θ (descida) e λ (subida), e projeção $\lambda \leftarrow \max(0, \lambda)$ a cada atualização. Estabilize com **OGDA** ou **Extragradient**.

Atualizações obrigatórias

Alternating steps. A cada iteração: (i) desça em θ ; (ii) suba em λ ; (iii) projete λ no semieixo positivo.

OGDA. Com $f(\theta, \lambda) = \mathcal{L}_B(\theta) - \lambda F(\theta; \theta_A)$:

$$\theta_{t+1} = \theta_t - \eta_\theta (2\nabla_\theta f_t - \nabla_\theta f_{t-1}), \quad \lambda_{t+1} = [\lambda_t + \eta_\lambda (2\nabla_\lambda f_t - \nabla_\lambda f_{t-1})]_+.$$

Extragradient. Dois estágios:

$$\begin{aligned} (\hat{\theta}, \hat{\lambda}) &= (\theta_t, \lambda_t) - (\eta_\theta, -\eta_\lambda) \odot \nabla f(\theta_t, \lambda_t), \\ (\theta_{t+1}, \lambda_{t+1}) &= (\theta_t, \lambda_t) - (\eta_\theta, -\eta_\lambda) \odot \nabla f(\hat{\theta}, \hat{\lambda}), \quad \lambda_{t+1} \leftarrow [\lambda_{t+1}]_+. \end{aligned}$$

Métricas e comparação

Além das métricas da Parte 1, registre a trajetória de λ . Compare OGDA e Extragradient em estabilidade das curvas, tempo até entrar em regime estável e desempenho final.

Relatório curto — Parte 2

1. Explique a formulação min-max e por que min e max competem;
2. compare OGDA vs. Extragradient visualmente e textualmente; relacione com os resultados da Parte 1.

Seção extra — Otimizadores alternativos

Além de OGDA/Extragradient para estabilizar o *jogo*, os alunos podem variar o otimizador subjacente em cada variável (ex.: Adam para θ e SGD para λ) para estudar efeitos em estabilidade, velocidade e retenção.

Como rodar

Parte 1 (penalização fixa, exemplo mínimo de deriva):

```
python simple_drift_example.py --epochsA 3 --epochsB 3 --lambda_reg 1e-3
```

Parte 2 (a implementar):

```
python game_minmax.py --ogda --theta_lr 1e-3 --lambda_lr 1e-2
```

```
python game_minmax.py --extragrad --theta_lr 1e-3 --lambda_lr 1e-2
```

Apêndice A — Exemplo mínimo: $\min_{\theta} \mathcal{L}_B(\theta) + \lambda F_{\text{drift}}(\theta; \theta_A)$

Nota: comentários em inglês para evitar problemas de acentuação dentro de **listings**.

```
# simple_drift_example.py
# Minimal example: train on Task A, then fine-tune on Task B with fixed lambda
# Objective: L_B(theta) + lambda * 0.5 * ||theta - theta_A||^2

import argparse, random
import numpy as np
import torch, torch.nn as nn, torch.nn.functional as F
import torch.optim as optim
from torch.utils.data import DataLoader, Subset
from torchvision import datasets, transforms

def set_seed(seed=42):
    random.seed(seed); np.random.seed(seed)
    torch.manual_seed(seed); torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False

class SplitMNIST(torch.utils.data.Dataset):
    def __init__(self, root, train, digits, transform, download=True):
        self.base = datasets.MNIST(root=root, train=train,
                                   transform=transform, download=download)
        self.idx = [i for i, (_, y) in enumerate(self.base) if y in digits]
    def __len__(self): return len(self.idx)
    def __getitem__(self, i): return self.base[self.idx[i]]

class MLP(nn.Module):
    def __init__(self, hidden=256, num_classes=10):
        super().__init__()
        self.fc1 = nn.Linear(28*28, hidden)
        self.fc2 = nn.Linear(hidden, num_classes)
    def forward(self, x):
        x = x.view(x.size(0), -1)
        x = F.relu(self.fc1(x))
        return self.fc2(x)

@torch.no_grad()
def evaluate(model, loader, device):
    model.eval(); ce = nn.CrossEntropyLoss()
    total, correct, loss_sum = 0, 0, 0.0
    for x, y in loader:
        x, y = x.to(device), y.to(device)
        logits = model(x); loss = ce(logits, y)
        loss_sum += loss.item() * x.size(0)
        pred = logits.argmax(1); correct += (pred == y).sum().item()
        total += x.size(0)
    return loss_sum/total, correct/total

def flatten_params(model):
    return torch.cat([p.detach().view(-1) for p in model.parameters()])

def train_task_A(model, loader, device, epochs=3, lr=1e-3):
    ce = nn.CrossEntropyLoss()
    opt = optim.Adam(model.parameters(), lr=lr)
    for _ in range(epochs):
        model.train()
        for x, y in loader:
            x, y = x.to(device), y.to(device)
            opt.zero_grad()
            loss = ce(model(x), y)
            loss.backward()
            opt.step()
        theta_A = flatten_params(model).clone().detach()
    return theta_A

def finetune_task_B_with_drift(model, loaderB, device, theta_A, lam=1e-3, epochs=3, lr=1e-3):
    ce = nn.CrossEntropyLoss()
    opt = optim.Adam(model.parameters(), lr=lr)
    for _ in range(epochs):
        model.train()
        for x, y in loaderB:
            x, y = x.to(device), y.to(device)
            opt.zero_grad()
            logits = model(x); L_B = ce(logits, y)
            theta = flatten_params(model)
            F_drift = 0.5 * torch.sum((theta - theta_A)**2)
            obj = L_B + lam * F_drift
            obj.backward()
            opt.step()

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument('--seed', type=int, default=42)
    ap.add_argument('--epochsA', type=int, default=3)
    ap.add_argument('--epochsB', type=int, default=3)
    ap.add_argument('--batch', type=int, default=128)
    ap.add_argument('--lr', type=float, default=1e-3)
    ap.add_argument('--lambda_reg', type=float, default=1e-3)
```

```

ap.add_argument('--device', type=str,
                default='cuda' if torch.cuda.is_available() else 'cpu')
args = ap.parse_args()

set_seed(args.seed)
device = torch.device(args.device)
tfm = transforms.ToTensor()

trainA = SplitMNIST('./data', train=True, digits={0,1,2,3,4}, transform=tfm)
trainB = SplitMNIST('./data', train=True, digits={5,6,7,8,9}, transform=tfm)
testA_full = datasets.MNIST('./data', train=False, transform=tfm, download=True)
idxA = [i for i, (_, y) in enumerate(testA_full) if y in {0,1,2,3,4}]
testA = Subset(testA_full, idxA)
testB_full = datasets.MNIST('./data', train=False, transform=tfm, download=True)
idxB = [i for i, (_, y) in enumerate(testB_full) if y in {5,6,7,8,9}]
testB = Subset(testB_full, idxB)

from torch.utils.data import DataLoader
loaderA = DataLoader(trainA, batch_size=args.batch, shuffle=True, num_workers=2)
loaderB = DataLoader(trainB, batch_size=args.batch, shuffle=True, num_workers=2)
loaderTestA = DataLoader(testA, batch_size=256, shuffle=False, num_workers=2)
loaderTestB = DataLoader(testB, batch_size=256, shuffle=False, num_workers=2)

model = MLP().to(device)
print('Training Task A...')
theta_A = train_task_A(model, loaderA, device, epochs=args.epochsA, lr=args.lr)
lossA0, accA0 = evaluate(model, loaderTestA, device)
print(f'After A:  lossA={lossA0:.4f} accA={accA0:.4f}')

print('Fine-tuning on Task B with fixed lambda (drift penalty)...')
finetune_task_B_with_drift(model, loaderB, device, theta_A,
                           lam=args.lambda_reg, epochs=args.epochsB, lr=args.lr)
lossA1, accA1 = evaluate(model, loaderTestA, device)
lossB1, accB1 = evaluate(model, loaderTestB, device)
print(f'After B:  accA={accA1:.4f}, accB={accB1:.4f}, forgetting={accA0-accA1:.4f}')

if __name__ == '__main__':
    main()

```